


## FACULTY OF TECHNOLOGY AND ENGINEERING

## DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

## DEPARTMENT OF INFORMATION TECHNOLOGY

## A.Y. 2026-27 [ODD]

# LAB MANUAL

# CSUC201: DSA



# Practical 3: Sorting Algorithms: Comparison-Based, Counting-Based, and Digit-Based Strategies.

**Problem 1**

A teacher has a stack of student answer sheets with marks written on them and needs to arrange them in order before entering grades. She tries three different methods: in the first, she repeatedly compares adjacent sheets and swaps them if they are out of order; in the second, she finds the lowest-marked sheet each time and places it at the front, in the third, she picks each sheet one by one and inserts it into its correct position among the already-arranged sheets. Implement all three methods and for each one, print the sorted order of marks.

**Code-**https://github.com/krishangpatel16-dotcom/FDSA-Batch-B/tree/main/Practical%203

**Output-**

```
enter number of marks:5
enter marks:
1
5
2
4
3
bubble sort:
1 2 3 4 5
selection sort:
1 2 3 4 5
insertion sort:
1 2 3 4 5
```

**Problem faced-**

-In this problem, I was confused about managing the loop conditions for the three sorting methods, but I overcame it by tracing each algorithm step by step.

**Conclusion:** All three methods correctly sort the marks, but they differ in efficiency: Bubble Sort and Selection Sort are generally slower for large lists, while Insertion Sort can be faster when the marks are already or nearly sorted.

**Problem 2**

A paint shop has buckets labeled with one of three colour codes — 0, 1, or 2 — but they are stored in a random order. The shop owner wants all 0s together, then all 1s, then all 2s, without using any extra storage. Given the list of colour codes, rearrange them in place and print the result.

**Code**-https://github.com/krishangpatel16-dotcom/FDSA-Batch-B/tree/main/Practical%203

**Output-**

```
enter number of colour codes:5
enter colour codes:
0
1
2
1
2
sorted colour codes:
0 1 1 2 2
```

**Problem faced-**

-In this problem, I found it difficult to sort the colour codes in place, but I overcame it by counting the number of 0s, 1s, and 2s first.

**Conclusion:** The array can be rearranged in-place so that all 0s come first, followed by 1s and then 2s, using constant extra space and a single pass, making it efficient for large lists.